
HerdLink

Real-time IoT telemetry & geofencing platform

A 4-week portfolio project demonstrating production-grade backend engineering at IoT scale.

Author	Asmar Ali — Senior Backend Engineer
Timeline	4 weeks (parallel to job applications)
Domain target	Halter (primary) • BrainRocket • general fintech / IoT
Scale target	1,000 simulated devices • <2s alert latency P95
Stack	NestJS, TypeScript, Kafka, Redis, TimescaleDB, MongoDB, MQTT
Deliverables	Live demo • README • ADRs • Grafana dashboards • load test

This document is the working blueprint for HerdLink — the project, the architecture, the technology choices, and the week-by-week build plan. The intent is to ship a portfolio piece that signals **day-one productivity** for senior backend roles at companies operating connected device fleets, high-throughput event systems, or real-time platforms. Halter is the primary target; the same engineering patterns translate directly to BrainRocket's iGaming and fintech workloads.

1. The Problem

Pasture-based livestock farming is operationally chaotic. A 500-cow herd grazing across 200 hectares of paddocks has to be moved, monitored, fenced, and health-checked daily — historically by humans on quad bikes, walking fences, and visually inspecting animals. This is labour-intensive, error-prone, and reactive.

Smart collars on livestock generate continuous telemetry — GPS, accelerometer, temperature, battery — that *should* enable proactive operations. The engineering challenges are non-trivial:

Scale	1,000 collars × 1 msg / 30s ≈ 3M events/day per farm. Multiply by 1,000 farms.
Connectivity	Rural cellular is patchy. Devices buffer offline and reconcile on reconnect.
Latency-sensitive	Cow breaching a fence onto a road needs an alert in <5s, not 5 min.
Geospatial at scale	Which of 10K devices is inside which of 500 polygons — every few seconds.
Mixed data shapes	Time-series telemetry, JSON configs, polygon geometry, audit logs.
Operational visibility	Live map view + historical analytics + alerting, all in one pane.

The problem in one sentence. Build a backend platform that ingests high-volume IoT telemetry from a simulated fleet of livestock collars, computes virtual fence breaches in real-time, alerts operators within seconds, and supports historical querying and live operational dashboards — with the resilience and observability expected of production systems.

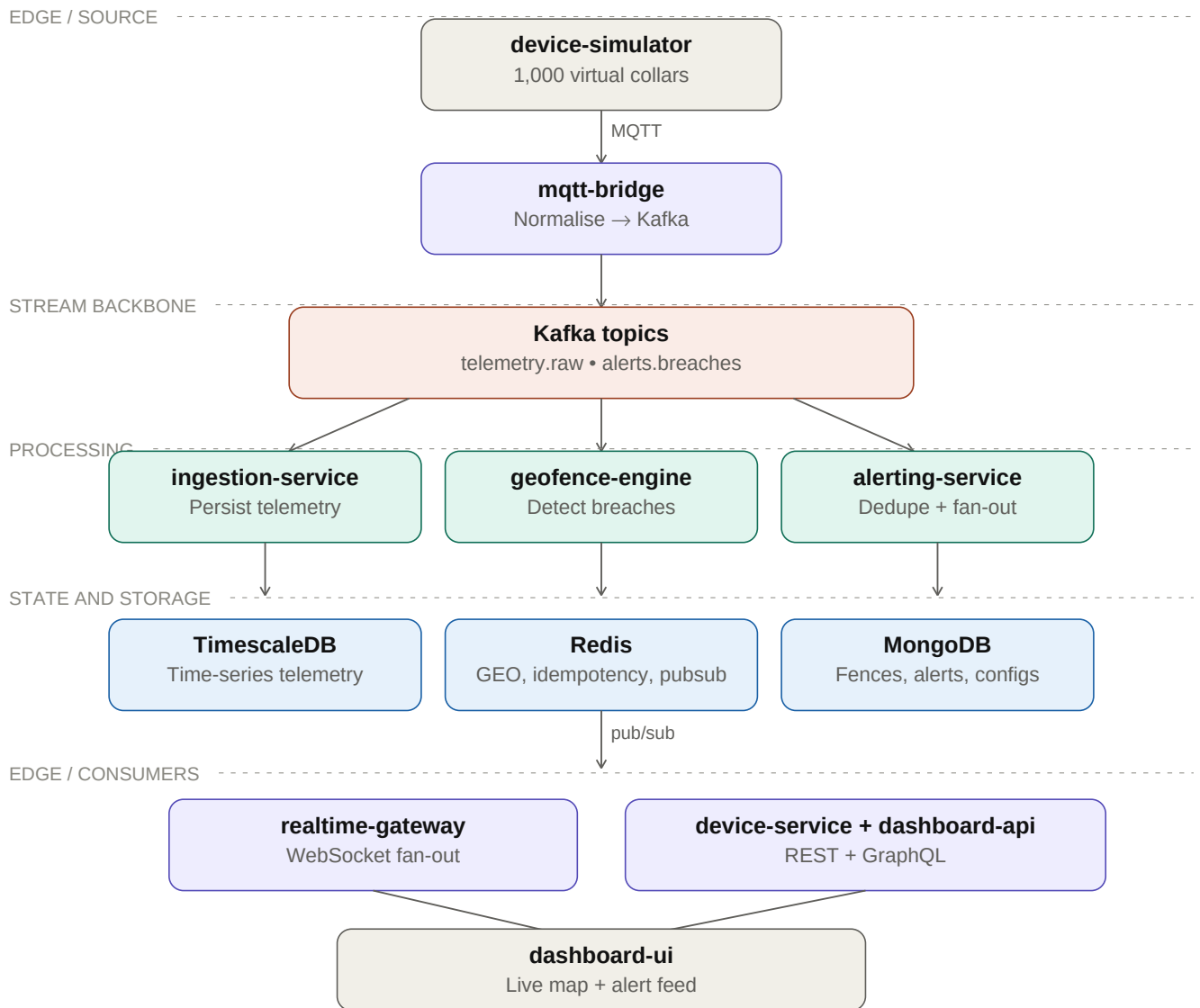
Why this is worth solving (the recruiter pitch)

“Halter operates one of the largest connected animal fleets in the world. I wanted to demonstrate I understood the engineering challenges before walking in the door — so I built HerdLink, a scaled-down version of the telemetry and geofencing pipeline a platform like Halter's depends on. Every architectural decision in this project mirrors trade-offs a real IoT-at-scale system has to make.”

The architecture generalises: Same event-driven pattern, swap collars for transactions, fences for fraud rules — and you have BrainRocket's iGaming workload. Swap cattle for vehicles, fences for service zones — and you have ride-hailing. The patterns transfer; the demo speaks to multiple targets simultaneously.

2. Architecture

Five tiers, single direction of data flow, every service does one thing well. The diagram below shows the full topology; each service, technology choice, and data flow is explained in the sections that follow.



Service responsibilities

#	Service	Responsibility
1	device-simulator	Spawns N virtual collars, generates realistic GPS walks + telemetry to MQTT
2	mqtt-bridge	Subscribes to MQTT, normalises payloads, publishes to Kafka
3	ingestion-service	Kafka consumer; writes raw telemetry to TimescaleDB; deduplicates
4	geofence-engine	Kafka consumer; checks each position against fences via Redis GEO; emits breach events
5	alerting-service	Consumes breach events; deduplicates; writes Mongo; publishes via outbox to Redis
6	device-service	CRUD for devices and geofences; REST + GraphQL
7	realtime-gateway	WebSocket server; subscribes to Redis pub/sub; fans out to dashboards
8	dashboard-api	BFF for dashboard UI; aggregates from other services via GraphQL
9	dashboard-ui	Live map + alert feed (React + MapLibre, deliberately thin)

The three critical flows

A — Telemetry ingestion (steady state)

simulator → MQTT → mqtt-bridge → Kafka [telemetry.raw] → ingestion-service (writes TimescaleDB) + geofence-engine (checks fences in Redis)

B — Geofence breach (the alert path)

geofence-engine → Kafka [alerts.breaches] → alerting-service (dedupes, writes Mongo) → outbox → Redis pub/sub → realtime-gateway → WebSocket → dashboard **Target: end-to-end < 2s.**

C — Operational query (the dashboard path)

dashboard → dashboard-api (GraphQL) → device-service + Redis (live positions) + TimescaleDB (history) + Mongo (alerts) → aggregated view

3. Technology Stack & Justification

Every choice below is defended on its merits versus realistic alternatives. The principle: pick the right tool for the actual workload, not the trendy or the familiar one. This section is here because in interviews this is the question.

NestJS + TypeScript

Eight microservices, one language, one consistent project structure. NestJS gives DI, modular architecture, and testing utilities out of the box — raw Express across eight services becomes eight subtly different conventions. TypeScript catches contract drift at compile time, especially around shared schemas. **Why not Go or Java?** Stack alignment with target companies (Halter, BrainRocket) and personal velocity. Both win.

Kafka

Three different services consume the same `telemetry.raw` stream independently. Adding a fourth consumer requires zero changes upstream. Replay capability is non-negotiable for fixing bugs in downstream consumers without losing data. **Why not RabbitMQ?** RabbitMQ excels at task queues; Kafka excels at event streams with multiple independent consumers and replay. The workload is the latter.

Redis

Three jobs, idiomatic for each: (1) sub-millisecond GEOSEARCH for fence breach detection — orders of magnitude faster than relational geospatial; (2) idempotency keys with TTL for effectively-once Kafka consumption; (3) pub/sub fan-out from alerting-service to the WebSocket gateway. Three responsibilities, but each is what Redis is genuinely good at — not over-using one tool.

PostgreSQL + TimescaleDB

Telemetry is time-ordered, append-heavy, and queried by time ranges. Timescale's hypertables give automatic time-based partitioning and order-of-magnitude faster range queries than vanilla Postgres. Choosing Timescale over InfluxDB means I keep relational integrity for device records (foreign keys, transactions) without running two database engines. **Single database, two workloads, both well-served.**

MongoDB

Geofences are GeoJSON polygons of varying complexity. Alerts have payloads that vary by alert type. Device configs are user-defined blobs. Forcing this into a relational schema means JSONB columns everywhere — which is just MongoDB with extra steps. **Polyglot persistence done deliberately, not accidentally.**

MQTT (Mosquitto)

MQTT is what real IoT devices use: low-bandwidth, persistent connection, QoS guarantees, last-will messages for disconnection detection. A telemetry platform that ingests over HTTP is one that hasn't met production constraints. Mosquitto in a container is one line of Docker Compose; the simulator publishes the way real collars would.

WebSockets (Socket.io)

Push alerts and live position updates to dashboards without polling. The realtime-gateway is the only stateful service in the system — keeping that concern isolated means the rest of the platform stays stateless and scalable.

GraphQL alongside REST

REST for simple CRUD on devices and fences. GraphQL for the dashboard, where a single query needs *device + last position + active alerts + nearby fences* — four resources, nested. REST would mean four round trips and stitching on the client. **Right tool for each surface; don't compromise either by picking one for both.**

Docker Compose

One command brings up the entire system. A recruiter who clones the repo gets instant gratification; a future engineer joining the team is productive in 15 minutes. Production would be Kubernetes; portfolio scope is Compose.

Avro schemas + Schema Registry

Producers and consumers evolve independently. Adding a field to telemetry doesn't break older consumers. Schema evolution is a real-world concern at the moment a system has more than one engineer; demonstrating it in a portfolio piece is a strong senior signal.

Pino + OpenTelemetry + Jaeger + Prometheus + Grafana

Observability is not an add-on. Structured logs with correlation IDs, distributed traces across all eight services, metrics on every consumer's lag and every flow's latency. **The system tells its own story** — and the screenshots of Grafana dashboards in the README are non-trivially what makes the project unignorable.

k6 (load testing)

Asserting performance with measured numbers, not vibes. The test simulates 1,000 devices for 10 minutes, asserts P99 alert latency under 5 seconds, and outputs results to `LOAD_TEST_RESULTS.md` in the repo. Recruiters skim; numbers in tables stop them.

Testcontainers + Jest

Unit tests for business logic. Integration tests with *real* Kafka, Redis, and Postgres in containers — no mocks where a real container costs nothing. End-to-end tests on the critical flows. **This is the test pyramid done correctly.**

4. 4-Week Implementation Plan

Principle: every week ends with a working, demonstrable system. No “we’ll integrate in week 4” — this plan builds vertical slices that work end-to-end from week 1, then deepens and hardens each week. The demo is the unit of progress. If a week’s demo doesn’t ship clean, the next week is re-scoped to recover.

WEEK 1

The Walking Skeleton

Goal. *A single device sends telemetry that flows end-to-end and shows up on a map. Every layer of the stack is touched.*

Build targets

- Monorepo scaffold (Nx or Turborepo) with NestJS service templates
- Docker Compose with: Mosquitto, Kafka (KRaft), Redis, PostgreSQL+TimescaleDB, MongoDB
- **device-simulator:** spawns one virtual collar publishing every 5 seconds
- **mqtt-bridge:** subscribes to MQTT, forwards to Kafka *telemetry.raw*
- **ingestion-service:** Kafka consumer, writes to TimescaleDB hypertable
- **realtime-gateway:** WebSocket server, broadcasts positions
- **dashboard-ui:** minimal MapLibre map with one moving dot
- Structured logging (pino) wired across all services
- README skeleton + architecture diagram committed

Engineering quality gates

- TDD from day one — failing test before any business logic
- Health check endpoints on every service
- Documented `.env.example` files
- Single *docker compose up* brings the whole system online

Demo at end of week

Open the dashboard. One dot moves across the map in real-time. Run `docker compose down && up` — the dot resumes from where it left off; telemetry persisted.

Why this milestone matters

The walking skeleton de-risks the entire project. By Friday, every integration point is proven. Every later week is *adding capability*, not *fighting the stack*. This is the most important milestone of the four.

WEEK 2

Scale, Geofences, and the Real Domain

Goal. *1,000 devices moving across realistic terrain, with virtual fences detecting breaches in real-time.*

Build targets

- **device-simulator** upgraded: 1,000 concurrent collars; realistic GPS random-walk; battery drain; configurable sample rate
- **geofence-engine** service: loads active fences from Mongo on startup; watches for changes; maintains live positions in Redis GEO; runs point-in-polygon on each incoming position; emits *alerts.breaches* events
- **device-service** with REST CRUD for devices and geofences (fences as GeoJSON)
- Dashboard UI: render 1,000 dots; draw fence polygons; click-to-create-fence
- Schema registry (Avro) for Kafka messages

Engineering quality gates

- Idempotency on the ingestion-service (dedupe on device_id + timestamp)
- Backpressure handling at Kafka consumers (manual offsets, max-in-flight limits)
- Integration tests using testcontainers (real Kafka, Redis, Postgres)
- ~80% coverage target on business logic
- Connection pool tuning (Postgres + Mongo)

Demo at end of week

1,000 cattle move in real-time on the map. Draw a fence polygon — within 2 seconds breaches appear in the console.

Why this milestone matters

This is where the project starts looking like a real platform. The geofence engine is the technical centrepiece: sub-millisecond Redis GEO queries against a live position set is exactly the detail that catches a senior engineer's eye.

WEEK 3

Alerting, GraphQL, and Production Patterns

Goal. *Full alerting pipeline with deduplication; the dashboard becomes a real ops tool; the system handles failures gracefully.*

Build targets

- **alerting-service**: consumes *alerts.breaches*; deduplicates (same device + same fence within N minutes); persists to MongoDB with state (NEW, ACKED, RESOLVED); publishes to Redis pub/sub
- **realtime-gateway** upgraded: subscribes to Redis pub/sub; authenticated WebSocket sessions (JWT); per-client subscriptions
- **dashboard-api** with GraphQL — query: device + last position + active alerts + nearby fences (the “stress test” query); mutations: ack alert, create/update fence
- Dashboard UI: live alert feed with severity colours; click cow → 24h history side panel; acknowledge alert button
- Outbox pattern in alerting-service for atomic Mongo-write + Redis-publish
- Distributed tracing across all services (OpenTelemetry → Jaeger)

Engineering quality gates

- End-to-end alert latency < 2s (instrumented + logged)
- Graceful degradation: kill dashboard-api → telemetry still ingests, alerts still fire
- Circuit breaker on Mongo writes in alerting-service
- Contract tests between services (shared schemas at minimum)

- Correlation IDs propagate from MQTT through to WebSocket push

Demo at end of week

Cow drifts across a fence. Within 2 seconds, a red alert appears with a sound. Click it: see the cow's last hour of movement. Acknowledge it. Open Jaeger: full trace from MQTT publish to WebSocket push.

Why this milestone matters

End-to-end alerting *with the right semantics* (deduplication, idempotency, graceful failure) is what separates a hobby project from a production-grade demo. The Jaeger trace screenshot alone is portfolio gold.

Goal. *The system tells its own story. Anyone cloning the repo sees metrics, runs the load test, and reads a polished README.*

Build targets

- Prometheus + Grafana stack in Docker Compose with pre-built dashboards (ingestion rate, consumer lag, alert latency P50/P95/P99, error rates, Redis hit rate)
- k6 load test: 1,000 devices for 10 minutes; asserts P99 alert latency < 5s; output committed as `LOAD_TEST_RESULTS.md`
- **README:** architecture diagram, why-this-exists pitch, why-each-tech-earns-its-place, quickstart (one command), demo script, screenshots, GIF
- Architecture Decision Records (ADRs) for the 5–6 most interesting choices
- Repo polish: GitHub Actions CI; badges; contributing guide
- 2-minute screen recording demo for LinkedIn

Engineering quality gates

- All Grafana dashboards load with data
- CI passes on a fresh clone
- README readable end-to-end in 5 minutes
- Test coverage report visible

Demo at end of week

Live demo. Load test passing. Grafana showing P99 alert latency < 5s under 1,000-device load. Architecture diagram and rationale for every choice. Recorded video walkthrough on LinkedIn.

Why this milestone matters

Week 4 is where the project becomes *unignorable*. Most portfolio projects skip observability and load testing. Having Grafana dashboards in the repo, with documented load test results, signals you operate the way senior engineers do — measuring, not guessing.

5. Engineering Quality Bar (Cross-Cutting)

These are the things that signal **senior** rather than *decent*. They run continuously across all four weeks — not as a separate phase. Each will have a corresponding ADR in docs/adr/.

- **Idempotent consumers.** Kafka delivers at-least-once. Idempotency keys (Redis-backed, 24h TTL) deliver effectively-once. No double-processed telemetry, no duplicate alerts.
- **Outbox pattern.** Atomic write to MongoDB + publish to Redis is impossible without a coordinator. Write to an outbox in the same Mongo transaction; a worker drains it to Redis. Standard pattern, correctly applied.
- **Schema evolution.** Avro schemas in a registry. Producers and consumers evolve independently. Adding a field doesn't break older consumers.
- **Backpressure handling.** Manual offset commits, max-in-flight limits, bounded consumer concurrency. Under load the system *slows*; it doesn't crash.
- **Graceful degradation.** Each service's failure mode is isolated and documented. Killing the dashboard-api does not stop ingestion or alerting.
- **Distributed tracing.** OpenTelemetry across every service. Correlation IDs propagate end-to-end. One trace ID, eight services, full causality.
- **Test pyramid.** Unit tests on logic; integration tests with testcontainers (real Kafka, Redis, Postgres in CI); end-to-end tests on critical flows. No mocks where a real container costs nothing.
- **Conventional commits + clean git history.** Squashed, logical commits. PR-style commit messages even on solo work. The git log itself is part of the portfolio.

6. Out of Scope & Risk Mitigations

Deliberately out of scope

A senior project is defined as much by what it doesn't try to do as what it does. Excluded explicitly so reviewers know they were considered, not forgotten:

- Authentication beyond simple JWT — this is not a SaaS
- Production multi-region deployment — architecturally feasible; not built
- ML-based animal behaviour detection — interesting, but unbounded scope
- Hardware integration — the simulator is the source of truth for telemetry shape
- Mobile clients — the dashboard is the demo surface
- Multi-tenancy — out of scope for a portfolio

Risk mitigations

Risk	Mitigation
Scope creep on the simulator (becoming a project of its own)	Hard cap: GPS random-walk + battery drain. No animal AI, no weather.
Frontend rabbit hole	Hard cap: map + sidebar + alert feed. No login, no settings page.
Stuck on infra in week 1	If Kafka/MQTT setup eats >2 days, pause and ask for help — don't burn the week solo.
Underestimating week 4 polish	The README and load test are the <i>product</i> . Not cleanup.

7. Closing

HerdLink is engineered to be the kind of artefact a hiring manager can read for two minutes and form a strong positive opinion. The architecture is real, the trade-offs are defended, the timeline is achievable, and the engineering quality bar is consistent with what senior backend engineers ship in production. The project is the message: **I can do your job on day one.**

Next step. Set up the monorepo skeleton and ship Week 1's walking skeleton.

Asmar Ali • Senior Backend Engineer • asmarali1929@gmail.com • asmarali.vercel.app